

■ Language Detector

Documentazione Tecnica

Sistema di rilevamento automatico della lingua di un testo
tramite modello di Machine Learning e interfaccia web

Progetto realizzato come lavoro di portfolio
Corso di AI Development

1. Introduzione e obiettivo del progetto

Language Detector è un'applicazione web che rileva automaticamente la lingua di un testo inserito dall'utente, tra cinque lingue supportate: **Italiano, Tedesco, Inglese, Francese e Spagnolo**.

L'utente scrive il testo in una casella di input e, in modo dinamico (senza bisogno di premere un bottone), il sistema mostra la lingua rilevata.

Il progetto è stato sviluppato con l'obiettivo di dimostrare competenze pratiche end-to-end in un flusso di lavoro di AI Development: dalla raccolta e pulizia dei dati, all'addestramento di un modello di Machine Learning da zero, fino all'esposizione del modello tramite un'API web e un'interfaccia utente.

1.1 Cosa fa il sistema

- Riceve un testo libero inserito dall'utente tramite interfaccia web.
- Classifica il testo come blocco unico, restituendo la lingua più probabile tra le 5 supportate.
- Mostra il risultato con nome della lingua e bandiera corrispondente.
- Segnala all'utente quando il testo inserito è troppo breve per garantire un'alta affidabilità.

1.2 Cosa il sistema NON fa (limiti di scope volutamente esclusi)

- Non rileva più lingue mescolate nello stesso testo con percentuali separate (es. "33% inglese, 66% italiano"): la scelta progettuale è stata quella di classificare il testo come unico blocco, vedi sezione 6 per la motivazione.
- Non riconosce lingue diverse dalle 5 supportate: il modello restituisce sempre una delle 5 lingue, anche per testi in altre lingue (non è stata implementata una categoria "lingua sconosciuta").

2. Stack tecnologico

Tecnologie, linguaggi e librerie utilizzate nel progetto:

Componente	Tecnologia	Ruolo nel progetto
Linguaggio	Python 3	Linguaggio principale per data processing, training e backend
IDE	PyCharm	Ambiente di sviluppo
Manipolazione dati	pandas	Caricamento, pulizia e campionamento del dataset
Download dati	requests	Download del corpus di frasi da Tatoeba
Machine Learning	scikit-learn	Feature extraction (TF-IDF) e modello di classificazione (Naive Bayes)
Serializzazione modello	joblib	Salvataggio/caricamento del modello allenato su disco
Backend / API	Flask	Server web che espone l'endpoint di predizione e serve l'interfaccia
Frontend	HTML, CSS, JavaScript	Interfaccia utente e chiamate dinamiche al backend (fetch API)
Fonte dati	Tatoeba.org	Corpus open di frasi multilingua usato per costruire il dataset di training

3. Struttura del progetto

Organizzazione delle cartelle e dei file:

```
language-detector/
■
■ ■■■ data/                                Dataset grezzi e puliti
■ ■■■ sentences.csv                        (dati grezzi scaricati da Tatoeba)
■ ■■■ dataset_clean.csv                  (dataset pulito e bilanciato, pronto per il training)
■
■ ■■■ src/                                Script Python di data processing e training
■ ■■■ download_data.py                  Scarica ed estrae il corpus Tatoeba
■ ■■■ explore_data.py                  Esplora il file grezzo (conteggi, anteprima)
■ ■■■ clean_data.py                    Filtra, pulisce, bilancia e salva il dataset finale
■ ■■■ train_model.py                   Allena il modello, lo valuta e lo salva su disco
■
■ ■■■ models/                           Modello allenato (output di train_model.py)
■ ■■■ language_model.joblib
■ ■■■ vectorizer.joblib
■
■ ■■■ app/                               Applicazione web
■ ■■■ app.py                           Backend Flask (API + serve la pagina)
■ ■■■ templates/
■ ■■■ index.html                       Interfaccia utente (HTML + JavaScript)
■ ■■■ static/
■ ■■■ style.css                        Stile grafico dell'interfaccia
■
■ ■■■ requirements.txt                 Elenco delle librerie necessarie
■ ■■■ README.md                       Istruzioni d'uso del progetto
```

4. Pipeline dei dati

4.1 Fonte dei dati

I dati grezzi provengono da **Tatoeba** (tatoeba.org), un database open e collaborativo di frasi tradotte in centinaia di lingue, ampiamente utilizzato in progetti di linguistica computazionale e NLP per la sua qualità e licenza libera. Il file utilizzato (*sentences.csv*) contiene oltre 13,5 milioni di frasi in tutte le lingue disponibili sulla piattaforma; da questo corpus sono state estratte solo le frasi nelle 5 lingue di interesse del progetto.

4.2 download_data.py

Scarica il file compresso *sentences.tar.bz2* da Tatoeba tramite richieste HTTP a blocchi (streaming), per evitare di saturare la memoria dato il peso considerevole del file. Successivamente estrae l'archivio nella cartella *data/*, ottenendo il file *sentences.csv* grezzo.

4.3 explore_data.py

Script diagnostico eseguito prima della pulizia vera e propria. Carica il file grezzo, conta il numero totale di frasi disponibili e, per ciascuna delle 5 lingue target, quante frasi sono disponibili. Questo passaggio è servito a verificare che il volume di dati fosse sufficiente rispetto all'obiettivo di 3.000 frasi per lingua, prima di procedere con la pulizia.

4.4 clean_data.py

Costruisce il dataset finale di training a partire dai dati grezzi, applicando i seguenti passaggi:

- **Filtro lingue:** mantiene solo le righe corrispondenti alle 5 lingue target (codici ISO 639-3 usati da Tatoeba: ita, deu, eng, fra, spa).
- **Rimozione valori mancanti:** elimina righe con testo assente.
- **Rimozione duplicati:** elimina frasi ripetute identiche.
- **Filtro per lunghezza:** mantiene solo testi tra 2 e 100 caratteri. La soglia minima è stata abbassata da 10 a 2 caratteri nel corso dello sviluppo, per includere anche parole ed espressioni brevi nel training (vedi sezione 6.2).
- **Campionamento bilanciato:** seleziona casualmente 3.000 frasi per ciascuna lingua, per un totale di 15.000 frasi, garantendo un dataset bilanciato (nessuna lingua sovra-rappresentata).
- **Riproducibilità:** il campionamento usa un seed fisso (`random_state=42`), così ogni esecuzione dello script produce lo stesso identico dataset.
- **Mescolamento finale:** le righe vengono mescolate per evitare che siano ordinate per lingua.

Output: *data/dataset_clean.csv*, con 15.000 frasi bilanciate tra le 5 lingue.

5. Modello di Machine Learning

5.1 train_model.py — panoramica

Questo script costituisce il cuore del progetto dal punto di vista ML: carica il dataset pulito, trasforma il testo in rappresentazione numerica, allena un modello di classificazione, ne valuta le prestazioni e lo salva su disco.

5.2 Suddivisione train/test

Il dataset viene diviso in un insieme di training (80%, 12.000 frasi) e un insieme di test (20%, 3.000 frasi), quest'ultimo mai utilizzato durante l'addestramento. Questa separazione è fondamentale per ottenere una stima onesta delle prestazioni del modello su dati mai visti, evitando il rischio di sopravvalutare l'accuratezza (overfitting). La suddivisione è **stratificata**: la proporzione tra le 5 lingue è mantenuta identica sia nel train che nel test set.

5.3 Feature extraction: TF-IDF su n-grammi di caratteri

Il testo viene trasformato in vettori numerici tramite **TfidfVectorizer** di scikit-learn, configurato per lavorare su **n-grammi di caratteri** (sequenze di 1-3 caratteri consecutivi) anziché su parole intere. Questa scelta è motivata dal fatto che ogni lingua presenta combinazioni di caratteri ricorrenti e caratteristiche (es. "sch" e "tz" in tedesco, "gli" e "zz" in italiano, "eau" in francese), che risultano più informative e robuste rispetto all'uso di parole intere, specialmente su testi brevi. Il peso **TF-IDF** (Term Frequency – Inverse Document Frequency) dà maggiore importanza agli n-grammi frequenti in una lingua specifica ma rari nelle altre, rendendoli più discriminanti ai fini della classificazione.

5.4 Algoritmo di classificazione: Multinomial Naive Bayes

È stato scelto **Multinomial Naive Bayes** come algoritmo di classificazione, per diversi motivi: è un algoritmo standard e consolidato per la classificazione di testo, è computazionalmente leggero e veloce sia in addestramento che in inferenza, e ottiene tipicamente ottime prestazioni su questo tipo di task senza necessità di ottimizzazioni complesse. Dato che i risultati ottenuti (vedi 5.5) sono già molto elevati, non è stato necessario ricorrere a modelli più complessi (es. reti neurali): la scelta di non aumentare la complessità del modello, quando non porta benefici misurabili, è essa stessa una decisione tecnica consapevole.

5.5 Risultati e valutazione

Il modello finale (allenato con soglia minima di 2 caratteri) ottiene un'accuracy del **99,07%** sul test set (3.000 frasi mai viste in training), con precision, recall e F1-score uniformi tra il 98% e il 100% per tutte e 5 le lingue, segno che il modello non presenta bias verso nessuna lingua in particolare né confonde sistematicamente lingue affini (es. italiano/spagnolo).

Lingua	Precision	Recall	F1-score
Tedesco (deu)	1.00	0.99	0.99
Inglese (eng)	0.99	0.99	0.99
Francese (fra)	1.00	0.99	0.99
Italiano (ita)	0.98	0.99	0.99
Spagnolo (spa)	0.99	0.98	0.99

Glossario metriche — Precision: percentuale di predizioni corrette tra tutte le volte in cui il modello ha assegnato quella lingua. Recall: percentuale di frasi di quella lingua correttamente identificate dal modello. F1-score: media armonica tra

precision e recall.

5.6 Salvataggio del modello

Il modello allenato e il vectorizer TF-IDF vengono serializzati su disco tramite **joblib** (*models/language_model.joblib* e *models/vectorizer.joblib*), così da poter essere caricati dal backend senza dover ripetere l'addestramento ad ogni avvio del server.

6. Scelte tecniche e motivazioni

6.1 Classificazione a blocco unico vs. percentuali per parola

In fase di progettazione è stata valutata la possibilità di classificare il testo parola per parola, restituendo una percentuale di composizione linguistica (es. "33% inglese, 66% italiano"). Questa opzione è stata scartata a favore della classificazione dell'intero testo come blocco unico, per un motivo tecnico preciso: le singole parole, essendo input molto brevi, offrono al modello un segnale statistico limitato (pochi n-grammi disponibili), portando a classificazioni meno affidabili e potenzialmente rumorose se aggregate in percentuali. È stato ritenuto preferibile offrire un singolo risultato affidabile su tutto il testo piuttosto che un risultato granulare ma meno accurato. Questa funzionalità resta un possibile sviluppo futuro (vedi sezione 8).

6.2 Gestione dei testi brevi

Durante i test è stato osservato che il modello, inizialmente allenato solo su testi di almeno 10 caratteri, classificava in modo poco affidabile parole molto corte (es. "Hola", classificato erroneamente come italiano anziché spagnolo). La causa è duplice: da un lato il modello non aveva mai visto, durante il training, esempi così brevi; dall'altro, testi molto corti contengono per loro natura pochissima informazione statistica (pochi n-grammi), rendendo la classificazione intrinsecamente più incerta indipendentemente dal modello utilizzato.

Sono state applicate due contromisure complementari:

- Il dataset di training è stato ricostruito includendo anche testi brevi (soglia minima abbassata da 10 a 2 caratteri), migliorando la capacità del modello di generalizzare su input corti.
- È stato aggiunto un avviso in interfaccia ("Testo breve: risultato meno affidabile") quando il testo inserito è sotto i 10 caratteri, così da informare onestamente l'utente sull'affidabilità attesa del risultato, invece di presentare sempre una risposta con apparente certezza assoluta.

Va sottolineato che questa rimane una **limitazione nota e intrinseca** del sistema: parole cortissime (3-4 caratteri) possono restare ambigue anche dopo il miglioramento, poiché il problema non è la quantità di dati di training ma la scarsità di informazione disponibile nell'input stesso.

6.3 Bandiere nell'interfaccia: da emoji a immagini

La prima versione dell'interfaccia utilizzava emoji-bandiera (es. 🇮🇹) per rappresentare visivamente la lingua rilevata. È stato riscontrato che i sistemi operativi Windows non compongono correttamente questo tipo di emoji (basate su coppie di "regional indicator symbols"), mostrando il codice paese testuale (es. "IT") anziché l'immagine della bandiera. Per garantire una resa visiva coerente indipendentemente dal sistema operativo dell'utente, le emoji sono state sostituite con immagini reali di bandiere, servite tramite il servizio gratuito flagcdn.com.

6.4 Perché Flask e non altri framework

Flask è stato scelto come framework backend per la sua leggerezza e semplicità, adatta a un progetto di dimensioni contenute con un solo endpoint funzionale principale. Offre inoltre pieno controllo su struttura HTML/CSS/JavaScript del frontend, a differenza di framework più "chiusi" orientati al prototyping rapido (es. Streamlit), permettendo di costruire un'interfaccia personalizzata.

7. Funzionamento dell'applicazione web

7.1 app.py — Backend Flask

Il backend espone due endpoint principali:

- **GET /** — restituisce la pagina HTML dell'interfaccia utente (index.html).
- **POST /predict** — riceve un JSON contenente il testo da analizzare, lo trasforma tramite il vectorizer TF-IDF, lo classifica con il modello Naive Bayes, e restituisce un JSON con il codice lingua, il nome leggibile della lingua, e un flag booleano che indica se il testo è troppo breve per un risultato pienamente affidabile.

Modello e vectorizer vengono caricati in memoria una sola volta all'avvio del server (non ad ogni richiesta), per garantire tempi di risposta rapidi.

7.2 index.html — Interfaccia e logica dinamica

La pagina contiene una casella di testo libera. Un listener JavaScript intercetta ogni modifica al testo e, tramite una tecnica di **debounce** (attesa di 400 millisecondi di pausa nella digitazione), invia il testo al backend solo quando l'utente smette di scrivere, evitando di generare una richiesta di rete ad ogni singolo carattere digitato. La risposta del server viene mostrata dinamicamente sotto la casella di testo, insieme alla bandiera della lingua rilevata e, se necessario, all'avviso di affidabilità ridotta.

7.3 style.css — Aspetto grafico

Definisce lo stile visivo dell'applicazione: sfondo con gradiente colorato, card centrale con ombra e bordi arrotondati, tipografia leggibile e stato di focus evidenziato sulla casella di testo.

8. Limiti noti e sviluppi futuri

Di seguito sono elencati i limiti attuali del sistema, tenuti volutamente fuori dallo scope di questa versione, insieme a possibili direzioni di sviluppo futuro:

- **Testi molto brevi (poche lettere):** restano una categoria intrinsecamente difficile da classificare con certezza, per la scarsità di segnale statistico disponibile.
- **Lingue non supportate:** il sistema classifica sempre il testo come una delle 5 lingue note, anche quando l'input è in una lingua diversa. Uno sviluppo futuro potrebbe introdurre una soglia di confidenza minima sotto la quale restituire "lingua non identificata".
- **Testi multilingua misti:** attualmente non è supportata la rilevazione di più lingue nello stesso testo con relativa percentuale di composizione. È una funzionalità tecnicamente realizzabile (classificazione a livello di singola parola con aggregazione dei risultati), scartata in questa versione per i motivi esposti in sezione 6.1, ma resta un possibile sviluppo futuro, eventualmente abbinato a modelli più robusti su input brevi.
- **Deploy:** l'applicazione è attualmente pensata per l'esecuzione in locale (ambiente di sviluppo). Un possibile sviluppo è la pubblicazione online tramite un servizio di hosting (es. Render, Railway, PythonAnywhere), con disattivazione della modalità debug di Flask per la messa in produzione.

9. Conclusioni

Il progetto Language Detector copre l'intero ciclo di sviluppo di un'applicazione basata su Machine Learning: raccolta e pulizia dei dati da una fonte pubblica, costruzione di un dataset bilanciato e riproducibile, addestramento e valutazione di un modello di classificazione testuale, esposizione del modello tramite un'API web, e realizzazione di un'interfaccia utente dinamica. Le scelte tecniche adottate sono state motivate e documentate, così come i limiti noti del sistema, con l'obiettivo di presentare un progetto completo, funzionante e tecnicamente consapevole dei propri confini.